

06/11/2023

3

Queremos añadir a nuestro ZeOS una funcionalidad para esperar a la finalización de los procesos:

```
int waitpid(int pid, int *error);
```

- Esta llamada bloquea al proceso actual en una lista de bloqueados hasta que se muera su proceso hijo con pid 'pid', almacena en la dirección de memoria 'error' el código de finalización de ese proceso y finalmente elimina el proceso.
- Si el proceso pid ya hubiera muerto, esta llamada no debe bloquearse.
- Esta llamada debe devolver error si el proceso pid no se encuentra dentro de la lista de sus hijos.
- Esta llamada a sistema tiene que usar la interrupción 130 para realizar la entrada a sistema (en lugar de los mecanismos ya implementados) y debe ejecutar la rutina de servicio directamente pues será el único servicio accesible mediante esta interrupción.

Los parámetros se pasarán por registro.

Este cambio implica modificar la llamada a sistema exit, que ahora será:

```
void exit(int error)
```

Esta llamada libera los recursos del proceso, pero no lo elimina sino que sólo lo marca como zombie (añadiendo el proceso actual a una lista de procesos zombies) y guarda el código de finalización error. Este código de finalización lo podrá consultar el padre del proceso con la llamada a sistema waitpid.

- Si el padre estuviera ya bloqueado esperando a la finalización de este proceso, esta función debe desbloquearlo.
- Si el proceso tuviera hijos, estos serán hijos del proceso idle que se encargará de su eliminación durante su ejecución. Nota: Puedes suponer que tienes implementadas las funciones siguientes: `void block(void)` : bloquea el proceso actual `void unblock(struct task_struct*pcb)`: desbloquea el proceso pasado como parámetro.

La solución tiene que ser genérica y funcionar de forma eficiente para cualquier número de procesos.

a) (0,5 puntos) Implementa el código del wrapper de la llamada `waitpid`

`libc_sys.S` (where wrappers are)

```
ENTRY(waitpid)
    pushl %ebp                # dynamic link
    movl %esp, %ebp
                                # no need to save edx/ecx
    movl 8(%ebp), %edx        # edx = int pid
```

```

movl 12(%ebp), %ecx # ecx = int* error
int $131           # SYSCALL custom for interrupt 131

cmpl $0, %eax      # check if error
jge waitpid_err_fi
pushl %eax         # pass error value to errno
call set_errno     # set errno to provided value
addl $4, %esp      # remove parameter from stack once call finished
movl $-1, %eax     # set return value to -1 as specified
waitpid_err_fi:
popl %ebp
ret

```

libc.h

```
int waitpid(int pid, int* error);
```

b) (1 punto) Implementa el código del handler de esta llamada a sistema.

entry.S

```

ENTRY(waitpid_system_call_handler)
    SAVE_ALL          # Save the current context
    call sys_waitpid  # Call service routine in sys.c
    movl %eax, 0x18(%esp) # Change the EAX value in the stack
    RESTORE_ALL       # Restore the context
    iret

```

c) (1 punto) Indica qué estructuras de datos se tienen que añadir y/o modificar. Añade el código necesario para inicializarlas.

- Tenemos que modificar la PCB (task_struct) y añadir un campo `int waitpid_pid`, que indicara a qué hijo está esperando.
- También añadir `int exit_error`, donde se guardará el código de error enviado a `void exit(int error)`.

sched.h

```

struct task_struct {
    int PID; /* Process ID. This MUST be the first field of the
struct. */
    page_table_entry * dir_pages_baseAddr;
    DWORD kernel_esp;
    struct list_head list; /* Anchor to the `ready` queue. */

    struct list_head children; /* List of children of this process. */

```

```

    struct list_head parent_list; /* Anchor to the parent's children list.
*/
    struct task_struct* parent; /* Parent of this process. */

    enum state_t state; /* Scheduling state of the process. */
    int quantum; /* Default quantum assigned to the process. */
    int pending_unblocks; /* If > 0, this process is blocked. */

    int waitpid_pid; // new
    int exit_error; // new
};

```

- Añadir el estado `ST_ZOMBIE` a `state_t`

`sched.h`

```

enum state_t {
    ST_RUN,
    ST_READY,
    ST_BLOCKED,
    ST_ZOMBIE // new
}

```

- Añadir una lista `zombies`

`sched.h`

```

extern struct list_head zombies;

```

`sched.c`

```

struct list_head zombies;

void init_sched() {
    // ...
    INIT_LIST_HEAD(&zombies);
}

```

- Si hace falta, añadir una lista de `blocked` (en realidad no es necesaria)

d) (1 punto) Modifica el código de la rutina `sys_exit`

```

void sys_exit(int error) {
    struct task_struct* task = current();

```

```

free_user_pages(task);

// add remaining children to idle process
struct list_head *element, *n;
list_for_each_safe(element, n, &task->children) {
    struct task_struct* children = list_entry(element, struct task_struct,
parent_list);
    list_del(element); // remove from old list
    list_add_tail(element, &idle_task->children); // add to new list
    children->parent = idle_task; // update parent
}

task->exit_error = error;
if (task->parent) {
    // if parent is waiting, unblock it
    if (task->parent->waitpid_pid == task->PID) {
        unblock(task->parent);
        dbg("[Child-K] Unblocked parent\n");
    }
    // add to zombies list
    task->state = ST_ZOMBIE;
    list_add_tail(&task->list, &zombies);
} else {
    update_process_state_rr(task, &freequeue);
}

sched_next_rr();
}

```

e) (1 punto) Implementa el código de la rutina de servicio sys_waitpid

```

int sys_waitpid(int pid, int* error) {
    struct task_struct* parent = current();
    struct task_struct* child_task = NULL;

    struct list_head* pos;
    list_for_each(pos, &parent->children) {
        struct task_struct* tmp = children_head_to_task_struct(pos);
        if (tmp->PID == pid) {
            child_task = tmp;
            break;
        }
    }
    // No child found with PID = pid
    if (child_task == NULL) return -ECHILD;

    // If child is not already dead, block
    if (child_task->state != ST_ZOMBIE) {
        parent->waitpid_pid = pid;
        block();
    }
}

```

```

// child has unblocked us, or is dead
*error = child_task->exit_error;

// finally remove process
child_task->parent = NULL;
child_task->PID = -1;
list_del(&child_task->parent_list);
update_process_state_rr( child_task, &freequeue);

return 0;
}

```

f) (0,5 puntos) ¿Es necesario modificar alguna otra llamada a sistema o parte del sistema para implementar por completo esta funcionalidad?

- Sí, cambiar la rutina `cpu_idle` para que elimine sus hijos una vez esten muertos.

`sched.c`

```

void cpu_idle(void)
{
    __asm__ __volatile__("sti": : : "memory");

    while(1) {
        struct list_head *element, *n;
        list_for_each_safe(element, n, &idle_task->children) {
            struct task_struct* child = list_entry(element, struct
task_struct, parent_list);
            if (child->state != ST_ZOMBIE) continue;

            // kill zombie
            child->parent = NULL;
            child->PID = -1;
            list_del(&child->parent_list);
            update_process_state_rr( child, &freequeue);
        }
    }
}

```

- Y `setIdt()` para añadir `setTrapHandler`

```

void setIdt() {
    // ...
    setTrapHandler(131, waitpid_system_call_handler, 3); // new
    // ...
}

```